

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Національний аерокосмічний університет

«Харківський авіаційний інститут»



Факультет Інтелектуальних систем управління

Кафедра Систем управління літальними апаратами

Лабораторна робота № 11

з дисципліни «Алгоритмізація та програмування»

на тему: «Розробка десктоп-застосунків в середовищі QtCreator»

XAI.301. G3.319a.01 ПР

Виконала: здобувачка 1 курсу, групи 319а
напряму підготовки (спеціальності)

“Електрична інженерія”

Вікторія БАБІНА

(ім'я і прізвище)

Прийняв: асис.,

_____ Євгеній ПЯВКА

МЕТА РОБОТИ

Ознайомитися з основами розробки програм з використанням QtDesigner і навчитися розробляти десктоп-застосунки із графічним користувацьким інтерфейсом для введення/виведення даних на мові програмування C++ в середовищі QtCreator.

ПОСТАНОВКА ЗАДАЧІ

Завдання 1. Вивчити алгоритм створення проекту Qt Widgets Application в середовищі QtCreator. Ознайомитись з налаштуваннями основних елементів для введення, виведення, компоновки форми і управління.

Завдання 2. Для вирішення завдання (табл.1) відповідно до варіанта:

2. Дано двозначне число. Знайти суму і добуток його цифр.

А. Спроекувати і реалізувати в конструкторі форм графічний інтерфейс програми з віджетами QLabel, QLineEdit і QPushButton. *Використати додаткові віджети і елементи компоновки.

В. *Додати і відлагодити програмний код для введення вхідних даних з перевіркою на коректність (використати QMessageBox для виведення сповіщень), обчислень і виведення результатів.

С*. Додати пункти меню у QMenuBar для зчитування вхідних даних і збереження результатів в файл з використанням стандартних діалогів для вибору файла.

Завдання 3. Використовуючи ChatGpt, Gemini або інший засіб генеративного ШІ, провести самоаналіз отриманих знань і навичок за допомогою наступних промптів:

1) «Ти - викладач, що приймає захист моєї роботи. Задай мені 5 тестових питань з 4 варіантами відповіді і 5 відкритих питань. Це мають бути завдання <середнього> рівня складності на розвиток критичного та інженерного мислення. Питання мають відноситись до коду, що є у файлі звіту, і до теоретичних відомостей, що є у файлі лекції»

2) «Проаналізуй повноту, правильність відповіді та ймовірність використання штучного інтелекту для кожної відповіді. Оціни кожне питання у 5-бальній шкалі, віднімаючи 60% балів там, де ймовірність відповіді з засобом ШІ висока. Обчисли загальну середню оцінку»

3) «Проаналізуй код у звіті, і додай опис і приклади коду з питань, які є в теоретичних відомостях, але не відпрацьовано у коді при вирішенні завдань»
 Проаналізуйте задані питання, коментарі і оцінки, надані ІІІ. Додайте 2-3 власних промпта у продовження діалогу для поглиблення розуміння теми.

ВИКОНАННЯ РОБОТИ

Завдання 1.

Алгоритм створення проєкту Qt Widgets Application у Qt Creator

1. Запустити програму Qt Creator.
2. Обрати пункт File → New Project.
3. Вибрати шаблон Qt Widgets Application.
4. Вказати назву проєкту та папку збереження.
5. Обрати комплект компілятора (Kit).
6. Створити головне вікно класу MainWindow.
7. Відкрити файл mainwindow.ui у Qt Designer.
8. Додати необхідні елементи інтерфейсу:
 - QLabel
 - QLineEdit
 - QPushButton
 - QGridLayout
9. Налаштувати властивості елементів.
10. Створити обробники натискання кнопок через механізм сигналів і слотів.
11. Додати програмний код у файл mainwindow.cpp.
12. Скомпілювати та запустити програму.

Основні елементи інтерфейсу, їх властивості та методи

1. QLabel

Використовується для відображення тексту або результатів обчислень.

Основні властивості:

- text — текст напису;
- alignment — вирівнювання;
- font — шрифт.

Основні методи:

- `setText()` — встановлення тексту;
- `text()` — отримання тексту.

2. QLineEdit

Використовується для введення даних користувачем.

Основні властивості:

- `placeholderText` — підказка;
- `maxLength` — максимальна довжина;
- `text` — поточний текст.

Основні методи:

- `text()` — отримання введеного тексту;
- `setText()` — встановлення тексту;
- `clear()` — очищення поля.

3. QPushButton

Використовується для виконання дії після натискання.

Основні властивості:

- `text` — напис на кнопці;
- `enabled` — доступність кнопки.

Основні методи:

- `clicked()` — сигнал натискання;
- `setEnabled()` — зміна доступності.

4. QMainWindow

Головне вікно застосунку.

Основні можливості:

- робота з меню;
- панелі інструментів;
- центральний віджет.

Основні методи:

- `show()` — показати вікно;

- `setWindowTitle()` — встановити заголовок.

5. QGridLayout

Використовується для розміщення елементів у вигляді таблиці.

Основні методи:

- `addWidget()` — додавання елемента;
- `setSpacing()` — відстань між елементами.

Переваги:

- автоматичне вирівнювання;
- зручне масштабування інтерфейсу.

6. QMessageBox

Використовується для виведення повідомлень про помилки.

Основні методи:

- `warning()` — попередження;
- `critical()` — критична помилка;
- `information()` — інформаційне повідомлення.

Завдання 2.

Вирішення задачі 2

Вхідні дані (ім'я, опис, тип, обмеження):

Value-двозначне число, введене користувачем, `int`, від 10 до 99 або від -99 до -10

Вихідні дані (ім'я, опис, тип):

Sum-сума цифр двозначного числа, `int`

Mult-добуток цифр двозначного числа, `int`

Message-повідомлення про помилку при неправильному введенні даних, `QString`

Алгоритм вирішення показано нижче

1. Користувач вводить двозначне число у поле `QLineEdit`.
2. Програма зчитує введене значення.
3. Виконується перевірка правильності введення числа.
4. Перевіряється, чи є число двозначним.
5. Виділяється перша цифра числа:

$$\text{first} = \text{abs}(\text{value}) / 10;$$

6. Виділяється друга цифра числа:

$$\text{last} = \text{abs}(\text{value}) \% 10;$$

7. Обчислюється сума цифр:

$$\text{sum} = \text{first} + \text{last};$$

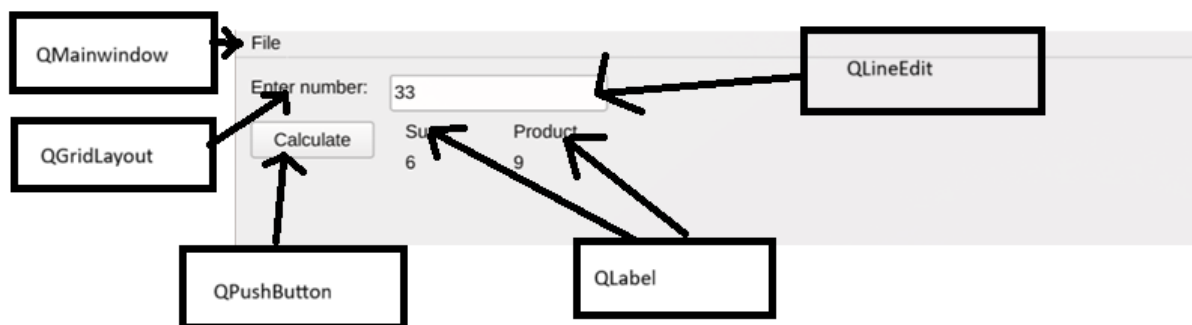
8. Обчислюється добуток цифр:

$$\text{mult} = \text{first} * \text{last};$$

9. Результати виводяться у елементи QLabel.

10. Якщо введені дані неправильні, виводиться повідомлення про помилку за допомогою QMessageBox.

Проект інтерфейсу показано на рис.1.



Код файла mainwindow.cpp:

```
#include "mainwindow.h"
#include "../ui_mainwindow.h"
#include <QString>
#include <QMessageBox>
#include <QFileDialog>

MainWindow::MainWindow(QWidget *parent)
    : QMainWindow(parent)
    , ui(new Ui::MainWindow)
{
    ui->setupUi(this);
}

MainWindow::~MainWindow()
{
    delete ui;
}
```

```

void MainWindow::on_calculate_clicked()
{
    bool ok;
    int value = ui->value->text().toInt(&ok);
    if (!ok) {
        QMessageBox::warning(0, "Error", "Number is not valid");
        return;
    }
    if (abs(value) < 10 || abs(value) > 99) {
        QMessageBox::warning(0, "Error", "Number should be in range [10,99]");
        return;
    }
    int first = abs(value) / 10;
    int last = abs(value) % 10;
    ui->sum->setNum(first + last);
    ui->mult->setNum(first * last);
}

void MainWindow::on_open_triggered()
{
    QString filePath = QFileDialog::getOpenFileName(this, "Open File",
QDir::currentPath(), "Текстові файли (*.txt);;Усі файли (*)");
    if (!filePath.isEmpty()) {
        QFile file(filePath);
        if (!file.open(QIODevice::ReadOnly | QIODevice::Text)) {
            QMessageBox::warning(0, "Error", "Can not open file!");
        } else {
            QTextStream in(&file);
            QString num, sum, mult;
            in >> num >> sum >> mult;
            ui->value->setText(num);
            ui->sum->setText(sum);
            ui->mult->setText(mult);
        }
    } else {
        QMessageBox::warning(0, "Error", "Unknown filename!");
    }
}

void MainWindow::on_save_triggered()
{
    QString filePath = QFileDialog::getOpenFileName(this, "Save File",
QDir::currentPath(), "Текстові файли (*.txt);;Усі файли (*)");
    if (!filePath.isEmpty()) {
        QFile file(filePath);
        if (!file.open(QIODevice::WriteOnly | QIODevice::Text)) {

```

```

        QMessageBox::critical(nullptr, "Error", "Can not open
file:\n"+file.errorString());
    } else {
        QTextStream out(&file);
        out << ui->value->text() << "\n";
        out << ui->sum->text() << "\n";
        out << ui->mult->text() << "\n";
    }
} else {
    QMessageBox::warning(0, "Error", "Unknown filename!");
}
}

```

Лістинг коду вирішення задачі 2 наведено в дод. А (стор. 10).

Екран роботи програми показаний на рис. Б.1.

Завдання 3.

1. В) Для введення тексту користувачем
2. С) toInt()
3. В) Виведення попередження про помилку
4. С) Для розміщення елементів інтерфейсу у вигляді таблиці
5. С) Знаходить першу цифру двозначного числа

1. Сигнали і слоти в Qt потрібні для взаємодії між елементами програми. Наприклад, коли користувач натискає кнопку, створюється сигнал `clicked()`, який викликає слот `on_calculate_clicked()`. У цьому слоті виконуються всі обчислення та виведення результатів.

2. Змінна `ok` потрібна для перевірки правильності введення числа. Якщо користувач введе букви або інші символи, то `toInt(&ok)` не зможе правильно перетворити значення, і `ok` стане `false`. Це допомагає уникнути помилок під час роботи програми.

3. Функція `on_calculate_clicked()` спочатку зчитує число з поля введення. Потім перевіряє, чи введене значення є правильним і чи воно двозначне. Після цього програма знаходить першу і другу цифру числа, обчислює їх суму та добуток і виводить результати у відповідні поля.

4. Функція `abs()` використовується для роботи з від'ємними числами. Вона перетворює число у додатне. Без `abs()` для від'ємних значень цифри могли б визначатися неправильно, і результати обчислень були б помилковими.

5. `QLabel` використовується для відображення тексту або результатів роботи програми. Користувач не може змінювати його вміст. `QLineEdit` призначений для

введення даних користувачем. У програмі `QLineEdit` використовується для введення числа, а `QLabel` — для показу суми та добутку цифр.

ВИСНОВКИ

У ході виконання лабораторної роботи було вивчено основи створення десктоп-застосунків у середовищі Qt Creator з використанням мови програмування C++. На практиці було закріплено навички створення графічного інтерфейсу, роботи з елементами `QLabel`, `QLineEdit`, `QPushButton`, а також реалізовано перевірку введених даних і роботу з файлами. Під час виконання роботи були отримані навички використання сигналів і слотів та виникли незначні труднощі з налаштуванням інтерфейсу і обробкою помилок введення даних.

ДОДАТОК А

Лістинг коду програми

```

#include "mainwindow.h"

#include <QApplication>

int main(int argc, char *argv[])
{
    QApplication a(argc, argv);
    MainWindow w;
    w.show();
    return QApplication::exec();
}

#include "mainwindow.h"
#include "../ui_mainwindow.h"
#include <QString>
#include <QMessageBox>
#include <QFileDialog>

MainWindow::MainWindow(QWidget *parent)
    : QMainWindow(parent)
    , ui(new Ui::MainWindow)
{
    ui->setupUi(this);
}

MainWindow::~MainWindow()
{
    delete ui;
}

void MainWindow::on_calculate_clicked()
{
    bool ok;
    int value = ui->value->text().toInt(&ok);
    if (!ok) {
        QMessageBox::warning(0, "Error", "Number is not valid");
        return;
    }
    if (abs(value) < 10 || abs(value) > 99) {
        QMessageBox::warning(0, "Error", "Number should be in range [10,99]");
        return;
    }
    int first = abs(value) / 10;
    int last = abs(value) % 10;
    ui->sum->setNum(first + last);
    ui->mult->setNum(first * last);
}

```

```

void MainWindow::on_open_triggered()
{
    QString filePath = QFileDialog::getOpenFileName(this, "Open File",
QDir::currentPath(), "Текстові файли (*.txt);;Усі файли (*)");
    if (!filePath.isEmpty()) {
        QFile file(filePath);
        if (!file.open(QIODevice::ReadOnly | QIODevice::Text)) {
            QMessageBox::warning(0, "Error", "Can not open file!");
        } else {
            QTextStream in(&file);
            QString num, sum, mult;
            in >> num >> sum >> mult;
            ui->value->setText(num);
            ui->sum->setText(sum);
            ui->mult->setText(mult);
        }
    } else {
        QMessageBox::warning(0, "Error", "Unknown filename!");
    }
}

void MainWindow::on_save_triggered()
{
    QString filePath = QFileDialog::getOpenFileName(this, "Save File",
QDir::currentPath(), "Текстові файли (*.txt);;Усі файли (*)");
    if (!filePath.isEmpty()) {
        QFile file(filePath);
        if (!file.open(QIODevice::WriteOnly | QIODevice::Text)) {
            QMessageBox::critical(nullptr, "Error", "Can not open
file:\n"+file.errorString());
        } else {
            QTextStream out(&file);
            out << ui->value->text() << "\n";
            out << ui->sum->text() << "\n";
            out << ui->mult->text() << "\n";
        }
    } else {
        QMessageBox::warning(0, "Error", "Unknown filename!");
    }
}

#ifdef MAINWINDOW_H
#define MAINWINDOW_H

#include <QMainWindow>

QT_BEGIN_NAMESPACE
namespace Ui {
class MainWindow;

```

```
}
QT_END_NAMESPACE

class MainWindow : public QMainWindow
{
    Q_OBJECT

public:
    explicit MainWindow(QWidget *parent = nullptr);
    ~MainWindow() override;

private slots:
    void on_calculate_clicked();

    void on_open_triggered();

    void on_save_triggered();

private:
    Ui::MainWindow *ui;
};
#endif // MAINWINDOW_H
```

ДОДАТОК Б
Скрін-шоти вікна виконання програми

The screenshot shows a window titled "File". Inside, there is a label "Enter number:" followed by a text input field containing the number "33". Below the input field is a button labeled "Calculate". To the right of the button, there is a table with two columns: "Sum" and "Product". The "Sum" column has the value "6" and the "Product" column has the value "9".

Sum	Product
6	9

Рисунок Б.1 – Екран виконання програми для вирішення завдання

ДОДАТОК В
Діалог з ШІ для самоаналізу

Тестові питання

1. Для чого використовується клас `QLineEdit` у програмі?

- A) Для виведення графіків
 - B) Для введення тексту користувачем
 - C) Для створення меню
 - D) Для відкриття файлів
-

2. Який метод використовується для перетворення тексту у ціле число?

- A) `toFloat()`
 - B) `toString()`
 - C) `toInt()`
 - D) `number()`
-

3. Яке призначення функції `QMessageBox::warning()`?

- A) Створення нового файла
 - B) Виведення попередження про помилку
 - C) Закриття програми
 - D) Обчислення результатів
-

4. Для чого використовується `QGridLayout`?

- A) Для збереження файлів
 - B) Для математичних обчислень
 - C) Для розміщення елементів інтерфейсу у вигляді таблиці
 - D) Для зміни кольору вікна
-

5. Що виконує оператор:

```
first = abs(value) / 10;
```

- A) Знаходить останню цифру числа
- B) Видаляє знак числа
- C) Знаходить першу цифру двозначного числа
- D) Обчислює квадрат числа

Відкриті питання

1.

Поясніть принцип роботи механізму сигналів і слотів у Qt. Як він використовується у вашій програмі?

2.

Чому у програмі використовується перевірка змінної `ok` після виконання `toInt(&ok)`? Які помилки це дозволяє уникнути?

3.

Опишіть послідовність дій функції `on_calculate_clicked()` від моменту введення числа до виведення результату.

4.

Навіщо у програмі використовується функція `abs()`? Що станеться без її використання для від'ємних чисел?

5.

Поясніть різницю між елементами `QLabel` і `QLineEdit`. У яких випадках використовується кожен із них?

Аналіз відповідей

Тестові питання

№	Оцінка	Аналіз відповіді	Ймовірність використання ШІ
1	5/5	Відповідь правильна та повна. Пояснення відповідає призначенню <code>QLineEdit</code> .	Низька
2	5/5	Правильно визначено метод <code>toInt()</code> .	Низька
3	5/5	Правильно пояснено призначення <code>QMessageBox::warning()</code> .	Низька
4	5/5	Відповідь правильно описує використання <code>QGridLayout</code> .	Низька
5	5/5	Правильно визначено призначення виразу для отримання першої цифри числа.	Низька

Відкриті питання

№	Оцінка	Аналіз відповіді	Ймовірність використання ШІ
1	4/5	Відповідь правильна, наведено приклад використання сигналів і слотів. Можна було детальніше описати механізм з'єднання сигналу та слоту.	Середня
2	5/5	Добре пояснено призначення змінної <code>ok</code> та перевірки введення.	Низька
3	5/5	Послідовність роботи функції описана правильно та логічно.	Низька

№	Оцінка	Аналіз відповіді	Ймовірність використання ШІ
4	4/5	Відповідь правильна, але коротка. Можна було навести приклад роботи без <code>abs()</code> .	Середня
5	5/5	Повністю пояснено різницю між <code>QLabel</code> і <code>QLineEdit</code> .	Низька

Загальна середня оцінка

[
 $\frac{5+5+5+5+5+4+5+5+4+5}{10}=4.8$
]

$\frac{5+5+5+5+5+4+5+5+4+5}{10}=4.8$

Середня оцінка: 4.8 / 5

Аналіз коду у звіті та додаткові приклади

У коді були відпрацьовані:

- створення графічного інтерфейсу;
- використання `QLineEdit`, `QLabel`, `QPushButton`;
- обробка сигналів і слотів;
- перевірка введення;
- робота з файлами;
- використання `QMessageBox`.

Проте деякі теоретичні можливості Qt у коді не використовувалися.

1. Використання `setPlaceholderText()`

Метод дозволяє додати підказку в поле введення.

Приклад:

```
ui->value->setPlaceholderText("Введіть двозначне число");
```

Це покращує зручність роботи користувача.

2. Використання `clear()`

Метод очищує поле введення.

Приклад:

```
ui->value->clear();
```

Може використовуватись після неправильного введення.

3. Використання `setWindowTitle()`

Метод встановлює назву головного вікна.

Приклад:

```
setWindowTitle("Лабораторна робота №11");
```

4. Використання `setEnabled()`

Дозволяє змінювати доступність кнопки.

Приклад:

```
ui->calculate->setEnabled(false);
```

Кнопка стане недоступною для натискання.

5. Використання `QMessageBox::information()`

Метод використовується для інформаційних повідомлень.

Приклад:

```
QMessageBox::information(this, "Info", "Файл успішно збережено");
```

6. Використання `QString::number()`

Метод перетворює число у текст.

Приклад:

```
QString text = QString::number(sum);
```

Це часто використовується для виведення результатів у `QLabel`.